

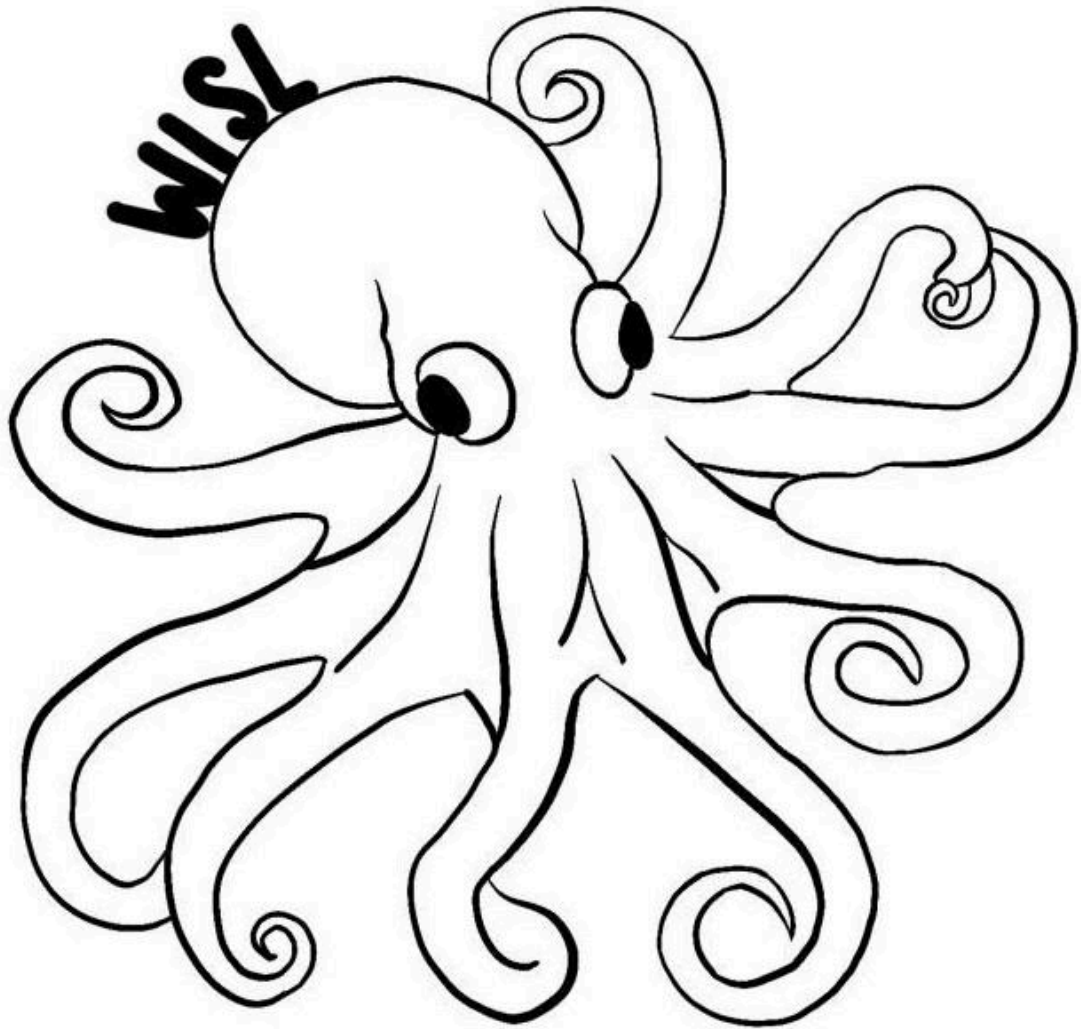
[WISL] [SDTH - Luma]

Singapore Defense Tech Hackathon - Luma

WISL

(Wartime Intelligence Simulation Layer)

[WISL] [SDTH - Luma]



Sua Isaac, Inessa Wong Manting, Sylvester Lim Huan Kiat

[WISL] [SDTH - Luma]

Introduction

WISL - Mission-memory and 3D simulations

Robotic failures are valuable data, not lost data. We seek to visualise the past to improve future mission planning.

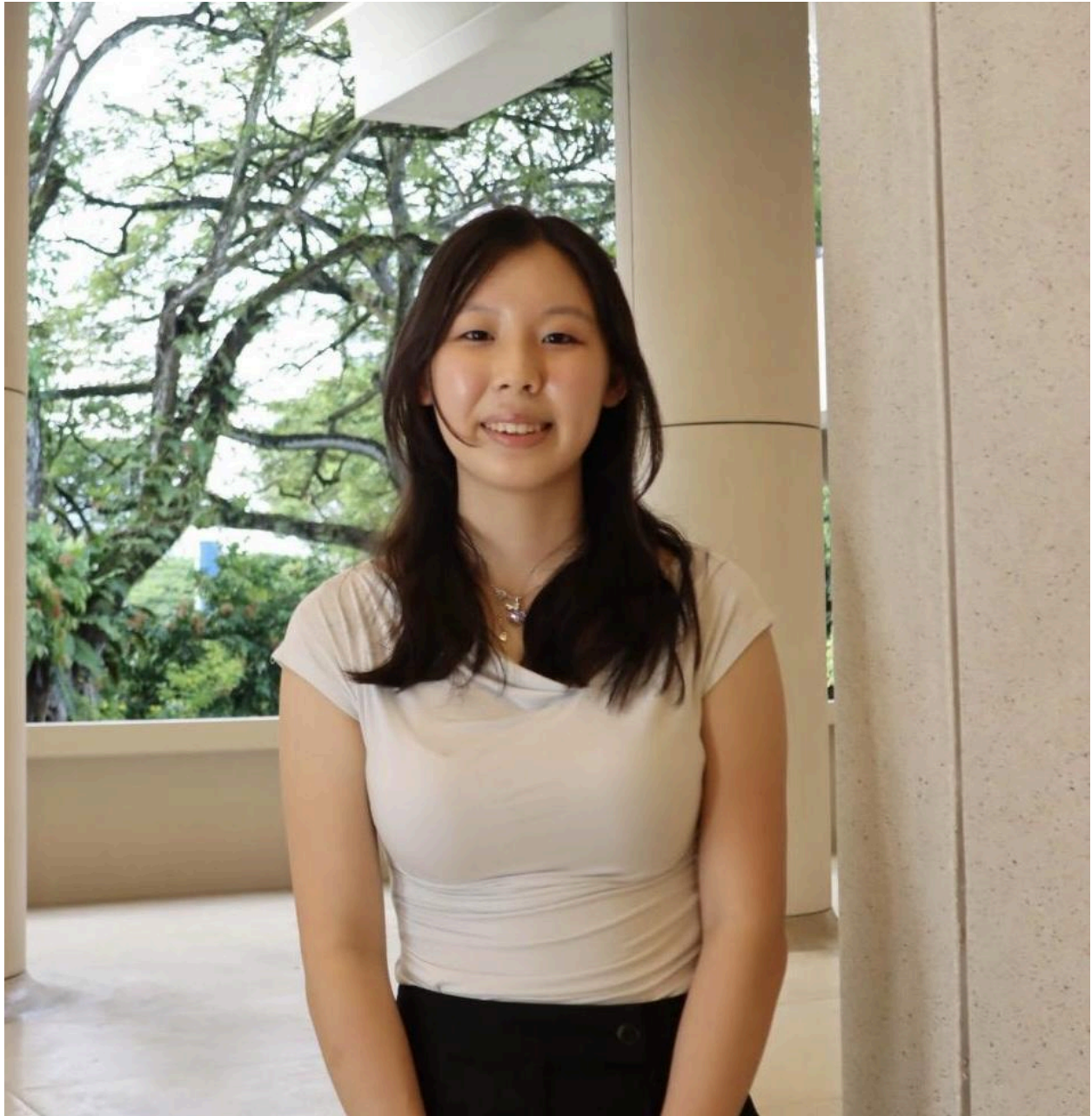
Sua Isaac



Research Intern at DSO National Laboratories. Experience in developing high voltage systems. Undergraduate of NUS DDP in Computer Engineering and Business Administration.

[WISL] [SDTH - Luma]

Inessa Wong Manting



Ex-trading intern at Golden Agri-Resources with experience in aligning fragmented time-series data & NUS Mathematics and CS student.

[WISL] [SDTH - Luma]

Sylvester Lim Huan Kiat - Pitcher, Back-End Developer



Software Engineering intern at IRAS & NUS CS student. Building agentic infrastructure using .NET core and Angular.

Scenario

Where does the problem appear?

Case 1: Wireless FPV Drone

The drone is wireless which may then result in failure data being lost. Consequently, we must then implement a way to retrieve or preserve this failure data.

On the other hand, the failure data might successfully be transmitted to the operator. However, this data might be too convoluted and thus be unreadable to the operator. As such, we must implement a simple way to visualise this error.

Case 2: Fibre Optic FPV Drone

Currently, flight telemetry is constantly transmitted back to the operator and stored in a data storage silo. As such, we must implement a way to submit this data telemetry to the cloud.

Shortfalls of the Status Quo

First, we must understand what are the current state-of-the-art, existing solutions. They are as follows:

AuterionOS for Telemetry Collection

AuterionOS is a commercial drone operating system built on PX4 and ROS2, designed to standardize how autonomous aerial systems collect and transmit data. Its telemetry pipeline captures flight parameters, such as attitude, GPS, IMU, battery state, motor outputs, and streams them via MAVLink to a ground control station.

Auterion Mission Control for Mission Planning

The Mission Control layer handles waypoint planning, geofencing, and live status monitoring.

- However, one challenge is that AuterionOS was designed for operational continuity, not forensic reconstruction. It tells us what the drone was doing, but not why it failed.

Nvidia's Isaac Sim 3D Simulation Platform for Visualizations

Isaac Sim is a high-fidelity robotics simulation platform built on the Omniverse USD framework. It supports physics-accurate environments, sensor simulation (LiDAR, cameras, IMU), and ROS2 integration, making it theoretically well-suited for replaying flight behavior.

- However, it was built primarily for pre-deployment robot training and testing, not for ingesting post-incident telemetry logs and reconstructing a failure frame-by-frame.

Now, we can look at constraints that make current approaches too slow, brittle, costly or hard to deploy.

Constraint	Elaboration
Mission data is fragmented	Drone operations generate data across multiple disconnected sources. To illustrate, this includes onboard SD cards, ground control telemetry streams, companion computer logs, and cloud sync services. However, this information is rarely timestamped or formatted consistently. A single failure event might have position data in one format, motor ESC logs in another, and camera metadata in a third. There is no unified data model that ties these into a coherent,

[WISL] [SDTH - Luma]

	queryable incident record. Reconstructing a failure requires manually correlating these streams, which is error-prone and time-consuming.
Failures are difficult to reproduce	Drone failures are often the product of rare, transient conditions. This may include a sensor dropout at a specific wind gust, a battery voltage sag under a particular load curve, a GPS multipath event in a specific location. Since these conditions are time and environment dependent, we cannot simply re-fly the mission to reproduce them. Without a structured replay mechanism that accurately restores the exact state of the vehicle and environment at the time of failure, the failure remains a one-time event that cannot be studied, validated, or used to train corrective behavior.
Parsing raw SD-card data requires engineering support	Raw flight logs from PX4 or ArduPilot are stored in binary formats (ULog, BIN/DataFlash) that require specialized tooling and domain knowledge to decode. A field operator or mission commander cannot extract meaningful insight without routing the SD card through an engineer who can run log parsers, filter the relevant signals, and interpret the output. This creates a bottleneck that delays tactical decision-making by hours or days. Moreover, this is time that matters when the same assets, with the same failure mode, are still being deployed.
Costly	When a failure is not fully understood, such as having root cause identified, conditions documented or corrective action validated, it enters the next deployment cycle as an unresolved risk. In fleet operations, this means the same failure mode will surface again across multiple assets, multiplying repair costs, mission losses, and potentially endangering personnel or infrastructure.

[WISL] [SDTH - Luma]

	The cost is not just the single incident. Rather, it is the entire downstream wave of repeated failures that a proper forensic-to-simulation pipeline would have prevented.
--	--

[WISL] [SDTH - Luma]

Proposed Solution

A mission-memory layer that provides real-time 3D failure visualizations to rapidly enhance tactical drone operations.

Core Capabilities

To realize this mission-memory layer, we will engineer a Unified Tactical Telemetry Cloud (UTTC). This architecture moves away from disparate mission planning software, creating an automated pipeline built on three core technical pillars as shown below:

Core pillar	Elaboration
Adaptive Data Ingestion Layer	Our system will dynamically secure data based on the asset's specific communication constraints: <i>Wireless Assets (High-Risk Data Loss)</i> We will implement an Edge-Preservation protocol. Onboard compute will run a lightweight anomaly detection algorithm. Upon sensing an imminent failure or critical event, it will package and burst-transmit a compressed, high-priority telemetry packet (a "Black Box" burst) before signal loss. <i>Fibre-Optic Assets (Continuous Volume)</i> We will deploy an automated cloud-sync gateway at local data storage silos, ensuring the continuous, secure uplink of high-fidelity flight telemetry without requiring manual engineering extraction.
Low-Overhead 3D Rendering Pipeline	We will construct a bespoke data pipeline using C++ and open graphics APIs (OpenGL/Vulkan). This hardware agnostic translation layer serializes standard UAV telemetry and plots spatial coordinates into a lightweight 3D coordinate space. This approach renders live flight paths on untextured, low-poly topological meshes, providing critical spatial awareness without the thermal or memory throttling associated with high-fidelity graphics engines.
Edge-Native SLM Telemetry	A major constraint in tactical drone ops is the fragmentation of proprietary vendor ecosystems. We will deploy a local relational database (SQLite) integrated with a 4-bit quantized Small Language Model (e.g., 3B parameters via ONNX runtime). This SLM

[WISL] [SDTH - Luma]

Normalization	acts as a dedicated text-processing engine. When an anomalous, unstructured error log or hexadecimal code is received from any vendor's drone, the SLM instantly translates it into a uniform system JSON schema, dynamically updating asset status flags across the fleet.
---------------	---

Why this differs

Edge-First Architecture

Unlike existing solutions that rely on stable uplinks to AuterionOS or heavy cloud processing, our system runs entirely on commercial off-the-shelf (COTS) edge compute. By utilizing an optimized C++ rendering pipeline and a local SQLite database, the system operates flawlessly under degraded communications. There is absolutely no cloud dependency or SaaS call-home required to visualize a drone failure.

Open Protocols & True Vendor Agnosticism

Currently, parsing SD-card data requires heavy engineering support and vendor involvement. Our interfaces are built entirely on open, documented standards (OpenGL/Vulkan). Furthermore, by deploying an edge-native Small Language Model (SLM) to normalize proprietary, unstructured error logs into a uniform JSON schema, operator-owned systems can connect, translate, and visualize failures from any drone manufacturer without requiring vendor permission or involvement.

Built for Singapore

Our architecture is inherently designed around Singapore's specific geography, density, threat profiles, and operating constraints. The reliance on edge-native processing and low-overhead spatial mapping directly addresses the realities of deploying drone swarms in high-density urban environments where high-bandwidth cloud connectivity is easily obstructed or contested.

3 Month Plan

The development roadmap prioritizes the deployment of the following features:

1. Low-overhead **data ingestion**
2. **3D rendering** pipeline
3. Edge-native **Small Language Model(s)** (SLMs) for heterogeneous log normalization and interface command routing

Weeks 1-4: Build

Focus

Construct the primary data pipeline to serialize and ingest standard UAV telemetry (e.g., MAVLink) and map spatial coordinates into a lightweight 3D coordinate space using C++ and open graphics APIs (OpenGL/Vulkan).

Operational Goal

Establish a hardware-independent data translation layer that renders live flight paths on untextured, low-poly topological meshes without the processing overhead of commercial game engines.

Deliverables

A local prototype software application capable of parsing historical flight logs or simulated stream arrays, updating position vectors inside a static 3D grid environment.

Success Criteria

Telemetry serialization and coordinate translation routines must execute under 500ms on a single thread of a standard mobile CPU, demonstrating zero coordinate accumulation drift over a test loop.

Weeks 5-8: Integrate

Focus

Deploy a local relational database instance (SQLite) and integrate an edge-native Small Language Model (e.g., 3B parameter model in 4-bit GGUF format via ONNX runtime) dedicated to schema normalization.

Operational Goal

Mitigate the issue of diverse vendors (and therefore distinct ecosystems) by using the SLM as a text-processing engine that ingests unformatted - proprietary hardware error logs from drone manufacturers and translates them into a uniform system JSON schema.

Deliverables

[WISL] [SDTH - Luma]

An integrated database and data-processing module. When an anomalous text log or hexadecimal error code is received, the system normalizes the record and updates the corresponding asset status flags in the database.

Success Criteria

The local SLM must parse unstructured vendor error descriptions and output structurally valid JSON matching the system schema within a few seconds of ingestion, maintaining zero structural formatting errors across a test suite of 100 sample logs.

Weeks 9-12: Demo

Focus

Optimize the rendering engine to process multi-agent telemetry frames concurrently and implement a secondary local SLM pipeline configured as an interface command router.

Operational Goal

Eliminate GUI processing bloat and simplify operator interaction. The operator inputs natural text commands, which the SLM maps directly to internal C++ function executions (e.g., coordinate isolation, playback offsets, rendering layer toggles).

Deliverables

A complete, standalone software package executing on the target low-spec edge hardware, featuring concurrent swarm rendering, automated log normalization, and text-based system orchestration.

Success Criteria

The system must process simultaneous telemetry streams for a swarm of agents without exceeding a reasonable amount of system memory allocation. The command router must correctly map operator text to the intended execution function latency.

Risks and Mitigations

Type of risk	Risk	Mitigation
Compute & Memory Contention	Hosting two/more localized SLM processing pipelines alongside a real-time 3D rendering engine might exhaust shared system RAM/VRAM on integrated graphics configurations.	Implement strict memory pooling and serialize SLM execution threads. The models will run using highly quantized 4-bit or 2-bit weights, and their context windows will be capped strictly at 512 tokens to limit peak memory footprints.
Algorithmic Variance	The SLMs can hallucinate. Malformed JSON strings during log normalization or incorrect function calls during command routing.	We mitigate this by structural determinism at the decoding layer. Apply strict grammar constraints (BNF/JSON-schema masking) during inference to force the token selection process to only generate syntactically valid commands or schemas.
Data Ingestion Bottlenecks	High-frequency telemetry updates from multi-agent arrays may saturate the single-threaded SQLite database write capacity, causing rendering pipeline lag.	Implement an in-memory ring-buffer queue for real-time telemetry rendering frame updates. Batch individual telemetry data writes to the local SQLite database disk storage at 5-second intervals to minimize I/O blockages.
Resource Dependency	A specialized team member's sudden absence or illness during the condensed 12-week build cycle halts	Enforce explicit architectural decoupling between the data ingestion layer, the SLM processing loop, and the graphics rendering subsystem via abstract interfaces. Maintain comprehensive internal

[WISL] [SDTH - Luma]

	development on a core technical component.	documentation and weekly code syncs.
Restricted Access to Real Failures Logs	Authentic military drone failure logs are often classified or highly restricted, leaving the team without real-world training data to validate models.	Use open-source physics engines to simulate high-fidelity physical and digital failure states. Generate dummy telemetry datasets to train and validate the system without traversing classification barriers.

Support and Integration

Asks From Organisers

1. *Edge Hardware*

Provision of a standard, low-spec ruggedized field tablet (e.g., Intel i5 processor with integrated graphics) or an NVIDIA Jetson Orin Nano development kit. This physical hardware is required to establish exact performance benchmarks for the rendering and model execution layers.

2. *UAV Platform Access*

Temporary access to a physical drone platform (PX4-based or commercial equivalent) and its ground control station to capture live telemetry transmission profiles and verify physical serial connection parameters.

3. *Unstructured Log Repositories*

Historical maintenance logs, error state readouts, and telemetry streams from multiple drone manufacturers so that we can calibrate the SLM data normalization training prompt arrays.

Asks From Partner Teams

1. *Topological Mapping Data*

3D terrain meshes or Elevation Model data of specified operational facilities to create a baseline of the memory allocation metrics within the graphics engine.

2. *Network Performance Benchmarks*

Real-world metrics detailing the typical bandwidth, lag, and packet drop rates of tactical battalion data links. We will use these specifications to simulate degraded network environments.

Closing Argument

Deploying this system transforms transient mission failures into permanent institutional assets, bringing modern software best practices directly to the tactical edge. Current architectures remain fractured by vendor-specific software stacks, heavy rendering pipelines, and a brittle dependency on remote cloud infrastructure to process mission data.

We resolve these bottlenecks through low-overhead C++ rendering and lightweight Small Language Models (SMLs). The system standardizes disparate hardware failures into actionable insights in real-time, bypassing proprietary vendor barriers without requiring high-spec graphics hardware.

Furthermore, replacing complex user interfaces with an edge-native command router arms operators with software that enables the query and replay of spatial mission records using intuitive text inputs. This delivers an easily scalable paradigm for the neo-battalion. It ensures that critical tactical adjustments can be derived and deployed directly on low-power field tablets within minutes of an asset failure. A loss that would otherwise mean a missing drone and a blind spot is instead transformed into an immediate tactical response. Software is the fastest adapting entity on the battlefield; this tool ensures our operators adapt just as quickly.